Script de soutenance — V1 (20 minutes)

> Support : `docs/Soutenance_PFE_V1_Transport_Scolaire.pptx` (32 slides : 25 de présentation + 7 d'annexe)
>
> **Comment lire ce document.** Pour chaque slide tu as trois blocs :
>
> - **À l'écran** — ce que le public voit, en une ligne.
> - **Ce que tu dis** — le texte à prononcer. Il est écrit en langage parlé, pas en langage écrit. Ne le lis pas mot à mot le jour J : lis-le 3 fois, puis raconte-le avec tes mots.
> - **Si on te demande** — la question probable et la réponse courte.
>
> **Règle d'or : une slide = une idée.** Si tu sens que tu t'étales, passe à la suivante. Ce qui reste dans la tête d'un encadrant, c'est la phrase-clé, pas le détail.

Comment gérer les 20 minutes

Bloc	Slides	Temps	Si tu es en retard
Cadrage	1 → 3	2 min 30	ne coupe rien, c'est ici qu'on comprend le sujet
Architecture	4 → 7	3 min 30	**coupe la slide 7** (sécurité)
Données et démarche	8 → 9	2 min	ne coupe rien, la slide 9 porte tout le mémoire
MS6 clustering	10 → 13	4 min	**raccourcis la 10**, jamais la 13
MS5 tournées	14	1 min	**coupe cette slide** si vraiment nécessaire
MS12 prédiction	15 → 17	3 min	**raccourcis la 15**, jamais la 17
MS11 anomalies	18 → 20	3 min	**raccourcis la 18**, jamais la 20
MS9 assistant	21 → 23	3 min	**coupe la 21**, garde 22 et 23
Bilan	24 → 25	1 min 50	ne coupe rien, c'est ta conclusion

Les 4 slides à ne jamais sacrifier : 9, 13, 17, 20. Ce sont celles où tu montres que tu sais juger ton propre travail. C'est ça qui fait la différence entre un bon projet et un bon mémoire.

BLOC A — CADRAGE (2 min 30)

Slide 1 — Page de titre

À l'écran : le titre, ton nom, celui de ton encadrant, et 4 chiffres à droite.

Ce que tu dis *(20 secondes — reste court, ne commente pas la page de titre)*

> Bonjour. Je vais vous présenter mon projet de fin d'études : un système de transport scolaire intelligent pour la ville de Meknès.
>
> Le titre complet, c'est « de la donnée géospatiale à la décision opérationnelle ». Je l'ai choisi parce que c'est exactement ce que fait le système : il part de positions GPS, et il en sort des décisions concrètes que des gens appliquent le matin même.

Slide 2 — Quatre décisions à prendre chaque matin

À l'écran : la carte des 300 élèves de Meknès à gauche, et 4 encadrés numérotés à droite.

Ce que tu dis *(1 min 10 — c'est la slide qui pose tout le sujet, prends ton temps)*

> Voici le point de départ. À gauche, ce sont 300 élèves réels, avec leurs vraies coordonnées GPS, répartis dans Meknès. Ils doivent tous arriver à la même école, l'école Al Massira.
>
> Chaque matin, quelqu'un doit répondre à quatre questions. Et c'est ça, mon sujet.
>
> **Première question : quel bus ?** Il faut répartir 300 élèves dans des bus de 50 places. Ce n'est pas juste « couper la carte en morceaux » : il faut que chaque groupe tienne dans un bus, et qu'il soit géographiquement serré. En science des données, ça s'appelle du clustering sous contrainte.
>
> **Deuxième question : dans quel ordre ?** Une fois qu'on sait qui monte dans quel bus, il faut décider l'ordre des 50 arrêts. C'est un problème d'optimisation classique, le problème du voyageur de commerce.
>
> **Troisième question : à quelle heure ?** Les parents veulent savoir quand le bus arrive. Il faut donc prédire une durée. C'est de la régression.
>
> **Quatrième question : qu'est-ce qui cloche ?** Un bus qui s'arrête trop longtemps, qui roule trop vite, qui sort de sa route. Le problème, c'est qu'on n'a pas d'exemples étiquetés d'incidents. C'est donc de la détection d'anomalies non supervisée.
>
> Quatre questions, quatre familles de problèmes différentes. C'est ce qui rend ce projet intéressant pour un master en science des données.

Si on te demande : *« Pourquoi Meknès ? »*

> Parce que ce sont de vraies coordonnées, un vrai réseau routier, et un vrai obstacle : l'oued Boufekrane coupe la ville en deux. Un projet sur des données inventées n'aurait pas eu ces difficultés-là.

Slide 3 — Objectifs et périmètre

À l'écran : 3 cartes (Décider / Prédire / Surveiller), un encadré bleu, et le fil rouge en bas.

Ce que tu dis *(1 minute)*

> Mes objectifs se résument en trois verbes.
>
> **Décider** : produire une affectation qui tient dans les bus, et des tournées calculées sur le vrai réseau routier — pas à vol d'oiseau.
>
> **Prédire** : donner une heure d'arrivée, avec une marge d'erreur chiffrée, et surtout savoir reconnaître quand le modèle sort de ce qu'il a appris.
>
> **Surveiller** : repérer ce qui est anormal en temps réel, sans avoir d'incidents étiquetés.
>
> Et je veux dire tout de suite une chose, avant qu'on me la demande : **il n'y a pas de deep learning dans ce projet.** Ce n'est pas un oubli. Mes données sont tabulaires, et j'ai 4 320 observations. J'ai comparé onze modèles, et un modèle linéaire régularisé bat XGBoost et LightGBM — même après réglage des hyperparamètres. Je vous montrerai les chiffres. Utiliser un réseau de neurones ici aurait été un choix de mode, pas un choix technique.
>
> Voilà le déroulé : je pose l'architecture rapidement, puis je passe le reste du temps sur les quatre briques d'intelligence artificielle.

Si on te demande : *« Un réseau de neurones n'aurait vraiment rien apporté ? »*

> Sur 4 320 lignes et 26 variables, non. Les arbres de décision, qui sont déjà plus puissants qu'un modèle linéaire, font moins bien. Un réseau ferait pire encore et serait impossible à expliquer à un exploitant.

BLOC B — ARCHITECTURE (3 min 30)

> **Consigne pour ce bloc** : va vite. L'architecture sert à montrer que tu maîtrises le système, pas à raconter la présentation. Si tu passes plus de 4 minutes ici, tu manges le temps de la science des données.

Slide 4 — Une architecture au service de quatre modèles

À l'écran : le schéma en couches. Les 5 services IA sont en bleu vif, tout le reste en gris-bleu.

Ce que tu dis *(1 min 20)*

> Voici l'architecture. Je l'ai volontairement simplifiée : le vrai système a plus de composants, mais je ne montre que ce qui structure.
>
> En haut, trois applications : une console web pour l'administration, et deux applications mobiles Flutter, une pour les parents, une pour les chauffeurs. L'application chauffeur est particulière : c'est le téléphone du chauffeur qui devient la source GPS du bus.
>
> Tout passe par une **passerelle unique**. C'est elle qui vérifie le jeton de connexion et qui limite le débit. Aucun client ne parle directement à un service.
>
> En dessous, treize microservices. Et je veux que vous remarquiez la couleur : **les cinq services en bleu, ce sont les services d'intelligence artificielle**. Ils sont écrits en Python avec FastAPI. Les autres, en gris, ce sont les services métier classiques en Java.
>
> Ce découpage n'est pas décoratif. Java est bon pour les transactions et la sécurité ; Python est le seul écosystème où j'ai scikit-learn, OR-Tools et pyproj. J'ai donc mis chaque partie là où elle est bonne.
>
> Tout en bas, les données : PostgreSQL avec PostGIS pour la géographie, Redis pour le cache des positions, et **OSRM** — c'est un serveur de calcul d'itinéraires que je fais tourner en local avec les données routières du Maroc. Ce dernier composant est central : c'est lui qui donne les vraies distances routières à trois de mes quatre briques.

Si on te demande : *« Pourquoi des microservices et pas une seule application ? »*

> Honnêtement, pour ce volume de données, une application unique aurait suffi. Le découpage se justifie par deux choses : les deux langages, et le fait que le clustering peut prendre plusieurs secondes sans devoir bloquer le reste du système.

Slide 5 — Synchrone pour décider, asynchrone pour réagir

À l'écran : trois bandes horizontales, une par chaîne d'événements.

Ce que tu dis *(1 minute)*

> Les services se parlent de deux manières.
>
> Quand on veut **une réponse**, c'est un appel direct : je demande, j'attends, j'obtiens.
>
> Quand on veut **signaler un fait**, c'est un message posté sur RabbitMQ. Celui qui poste ne sait pas qui va lire, et il n'attend personne.
>
> Je ne vous montre que les trois chaînes qui portent de l'intelligence artificielle.
>
> **La première** : le bus envoie sa position, mon service de prédiction la reçoit, recalcule l'heure d'arrivée, et quand elle passe sous dix minutes il émet un événement qui déclenche la notification aux parents. Ce qui est important ici, c'est que **le modèle n'est pas interrogé par un humain** : il est déclenché par le flux GPS, tout seul.
>
> **La deuxième** : la même position part aussi vers mon service de détection d'anomalies, qui applique trois règles instantanées.
>
> **La troisième** : quand un élève est inscrit, le clustering est relancé automatiquement. Et ça, ça crée une exigence forte dont je reparlerai : il faut que le résultat soit **stable**, sinon un enfant change de bus parce qu'un autre s'est inscrit.

Slide 6 — Le chemin d'un point GPS

À l'écran : quatre étapes avec des flèches, puis trois cartes de chiffres.

Ce que tu dis *(40 secondes — vite)*

> Concrètement, voici ce qui se passe pour un seul point GPS.
>
> Le téléphone du chauffeur envoie sa position toutes les cinq secondes. Le service de localisation la range dans Redis, la diffuse en WebSocket aux cartes ouvertes, et la publie sur le bus de messages. Le service de prédiction recalcule l'heure d'arrivée. Et si elle est proche, le parent reçoit une notification.
>
> J'ai mesuré la chaîne complète : **trois secondes**, avec un bus à 2 200 mètres de l'école.
>
> Les deux autres chiffres sont importants. J'ai dû ajouter des garde-fous, parce que sans eux le système devenait inutilisable. Regardez le troisième : avant correction, un parent recevait **150 notifications identiques en trois minutes**. La raison est bête : un bus garé à l'école continue d'envoyer sa position, donc la règle « le bus est arrivé » se déclenchait à chaque point. J'ai remplacé ça par un verrou : on annonce l'arrivée **au moment où on entre dans la zone**, une seule fois.

Slide 7 — Autoriser n'est pas cadrer la portée

> **C'est la slide à couper en premier si tu es en retard.**

À l'écran : deux colonnes (Couche 1 / Couche 2) et un encadré orange en bas.

Ce que tu dis *(30 secondes)*

> Un mot sur la sécurité, parce que ça prépare la partie sur l'assistant conversationnel.
>
> Il y a deux couches, et beaucoup de projets confondent les deux.
>
> La première, c'est **le rôle** : es-tu administrateur, chauffeur ou parent ?
>
> La deuxième, c'est **la portée** : d'accord, tu es chauffeur — mais est-ce que cet élève-là est dans **ton** bus ?
>
> J'avais vérifié 45 combinaisons de rôles et de routes, et tout était correct. Puis j'ai refait une revue centrée non plus sur les rôles mais sur **les interactions entre acteurs**. J'ai trouvé quinze failles. La plus parlante : un chauffeur avait le bon rôle pour pointer la présence de l'élève d'un collègue — ce qui envoyait une notification au parent d'une autre famille.
>
> La leçon : ce sont deux couches distinctes, et il faut tester les deux.

BLOC C — DONNÉES ET DÉMARCHE (2 min)

Slide 8 — Trois natures de données

À l'écran : un tableau à trois lignes, et un encadré bleu en bas.

Ce que tu dis *(1 min 10 — dis-le calmement, c'est un moment d'honnêteté)*

> Je veux être clair sur mes données, avant qu'on me pose la question.
>
> J'ai **trois natures de données différentes**, et elles n'autorisent pas les mêmes conclusions.
>
> **Les données réelles** : les 300 élèves et leurs coordonnées, et le réseau routier de Meknès via OSRM. Là, mes conclusions sont valides — mais sur **une seule instance** : une école, 300 élèves. Je n'ai pas testé le passage à l'échelle.
>
> **Les données semi-réelles** : pour la prédiction d'heure d'arrivée, j'ai généré 4 320 trajets sur 180 jours d'école. La **durée de base de chaque trajet vient d'OSRM**, donc du vrai réseau. Ce que j'ai simulé, ce sont les facteurs : la météo, le trafic, le ramadan, le jour de souk — avec des coefficients tirés de la littérature en transport.
>
> **Les données synthétiques contrôlées** : pour la détection d'anomalies, j'ai injecté des anomalies moi-même. Et là il faut comprendre pourquoi : la détection d'anomalies est **non supervisée**, le modèle n'apprend pas les étiquettes. Mais sans étiquettes, **je ne pourrais calculer aucune métrique**. Je les crée donc uniquement pour évaluer.
>
> Et je vais plus loin, avec l'encadré du bas. Pour la prédiction, mes données sont produites par une **formule multiplicative**. Or un modèle linéaire sur variables indicatrices retrouve presque exactement une formule multiplicative. Donc **une partie de la victoire d'ElasticNet est inscrite dans mon générateur**. Le protocole reste valable, mais je ne peux pas affirmer que ça généraliserait sur des durées GPS réelles.

Si on te demande : *« Vos résultats valent-ils quelque chose si les données sont simulées ? »*

> Ce qui vaut, c'est le protocole : le découpage temporel, le bootstrap, l'ablation, la comparaison à des baselines. Ces mesures-là seraient identiques sur des données réelles. Ce qui ne vaut pas, c'est le chiffre absolu de MAE. C'est pourquoi ma première perspective est de réentraîner sur l'historique que le système collecte lui-même.

Slide 9 — Comment je juge mes propres modèles

> **Slide capitale. Ne la coupe jamais.** C'est elle qui transforme cinq notebooks en une démarche.

À l'écran : les 7 règles, les règles 2 et 7 surlignées en bleu.

Ce que tu dis *(50 secondes)*

> Après avoir livré les cinq briques, j'ai fait quelque chose d'inhabituel : **je les ai toutes re-benchmarkées, en écrivant d'abord les règles du jeu.** Sept règles.
>
> Je vous en commente deux, celles qui ont trouvé les vrais problèmes.
>
> **La règle 2 : l'algorithme qu'on déploie doit figurer dans la comparaison.** Ça paraît évident. Pourtant mon clustering comparait dix algorithmes… et celui qui tourne réellement en production n'était dans aucun des dix.
>
> **La règle 7 : il faut consommer la même source de données que le service.** Exécuter la vraie fonction ne suffit pas. Mon benchmark appelait bien la bonne fonction, mais en lui donnant les élèves triés depuis un fichier, alors que le service les reçoit dans l'ordre de la base de données. Ça m'a fait publier un résultat faux.
>
> Le résultat de cette campagne : **dans les cinq briques sur cinq, la mesure a contredit ou nuancé une affirmation de mon propre dossier.** Vous allez en voir quatre exemples dans la suite.

BLOC D — MS6 : CLUSTERING (4 min) — le cœur

Slide 10 — Ce n'est pas un KMeans()

À l'écran : la carte avec l'école et l'oued, et trois encadrés à droite.

Ce que tu dis *(1 minute)*

> Première brique : répartir les élèves dans les bus.
>
> On pourrait croire qu'un `KMeans` suffit. Trois contraintes disent le contraire.
>
> **Un : la capacité est une contrainte dure.** Un groupe de 53 élèves n'est pas « un peu trop grand », il est **inutilisable**. Un bus a 50 places, point. Or KMeans ne sait pas faire ça.
>
> **Deux : la compacité n'est pas un score esthétique.** Quand je dis qu'un groupe est « compact », je parle du **temps que l'enfant passe dans le bus le matin**. C'est une métrique métier, pas mathématique.
>
> **Trois : la stabilité.** Le clustering est rejoué à chaque inscription. Si le résultat change à chaque fois, des enfants changent de bus sans raison. Pour une famille, c'est inacceptable.
>
> Et le terrain ajoute sa difficulté : vous voyez sur la carte l'**oued Boufekrane**. Deux élèves de part et d'autre sont proches à vol d'oiseau, et très loin en voiture. J'ai aussi 5 % d'élèves isolés, qui ne rentrent dans aucun quartier.

Slide 11 — Deux décisions de préparation qui changent tout

> **La slide « je suis data scientist et pas juste utilisateur de sklearn ».** Prends ton temps.

À l'écran : deux encadrés bleus, et en dessous le graphique route vs vol d'oiseau.

Ce que tu dis *(1 min 10)*

> Avant tout algorithme, il y a deux décisions de préparation. Aucune des deux ne relève de scikit-learn : elles relèvent de la géométrie du terrain.
>
> **Première décision : projeter, et surtout ne pas normaliser.**
>
> Le réflexe habituel, c'est de prendre la latitude et la longitude et de leur appliquer un `StandardScaler`. C'est **faux**, et voilà pourquoi. À la latitude de Meknès, 33,9 degrés Nord, un degré de longitude ne fait pas la même distance qu'un degré de latitude. Si vous normalisez les deux de la même façon, **vous déformez la carte**. Vos distances ne veulent plus rien dire.
>
> J'ai donc projeté en **UTM zone 30 Nord**, avec la bibliothèque pyproj. Résultat : mes coordonnées sont **en mètres**. Et ça change tout, parce que mon score de silhouette devient interprétable : je peux dire « la compacité médiane est de 734 mètres », pas « 0,73 unité sans nom ».
>
> **Deuxième décision : mesurer la route, pas le vol d'oiseau.**
>
> Regardez le graphique du bas. J'ai construit une **matrice de distances routières de 300 sur 300**, en interrogeant OSRM. C'est-à-dire 90 000 trajets réels calculés sur le réseau de Meknès.
>
> Le résultat mesuré : **le vol d'oiseau sous-estime le trajet réel d'un facteur 1,5 à 1,7.** Et l'oued est exactement le genre d'obstacle qui crée cet écart.

Si on te demande : *« 300 fois 300, ça ne fait pas trop de requêtes ? »*

> OSRM plafonne à 100 coordonnées par requête. J'assemble donc la matrice **par blocs de 50 sur 50**, avec six requêtes en parallèle. Et je la mets en cache dans un fichier, donc elle n'est calculée qu'une fois.

Slide 12 — Le pipeline déployé

À l'écran : quatre étapes avec des flèches, puis cinq chiffres de production.

Ce que tu dis *(1 minute)*

> Voici ce qui tourne réellement, en quatre étapes.
>
> **Un — DBSCAN.** Je commence par isoler les élèves dispersés, ceux qui sont loin de tout. Il y en a 17. Les mettre dans un groupe fausserait toute la mesure.
>
> **Deux — la courbe de Pareto.** Et c'est un point que je défends : **je ne fixe jamais le nombre de bus.** Je trace une courbe qui met en face la qualité des groupes et le nombre de bus, et je laisse la courbe arbitrer. Le minimum théorique est 300 divisé par 50, donc 6. Le système en choisit 7.
>
> **Trois — les k-médoïdes.** Contrairement à KMeans qui calcule une moyenne, le médoïde est **un vrai élève**, donc un vrai point du réseau routier. Une moyenne de coordonnées pourrait tomber au milieu de l'oued.
>
> **Quatre — un programme linéaire.** C'est l'étape la plus importante. Au lieu d'affecter chaque élève au centre le plus proche, je résous un **problème de transport** avec scipy : je minimise la distance totale, sous la contrainte que chaque groupe fasse au plus 50. Et là est le point : la capacité n'est pas **vérifiée après coup**, elle est **garantie par la construction**. Il est mathématiquement impossible que la solution dépasse 50.
>
> Les chiffres de production sont en bas : 7 bus, silhouette 0,4256, compacité médiane 734 mètres, taille maximale 50.
>
> Et je veux commenter cette silhouette de 0,43, parce que dans les cours on apprend qu'il faut viser 0,5. **C'est un arbitrage, pas un mauvais score.** Sans la contrainte de capacité, KMeans monte à 0,4636 — mais il produit un groupe de **66 élèves**, donc inutilisable. J'ai chiffré ce que coûte le réalisme : **3,4 %**. Je préfère 3,4 % de silhouette en moins et une solution qui marche.

Slide 13 — Ce que l'ablation a révélé, et la limite que j'ai trouvée

> **Slide à ne jamais couper.** C'est ici que tu montres ta démarche scientifique.

À l'écran : un tableau de 6 méthodes à gauche, un encadré orange à droite.

Ce que tu dis *(50 secondes)*

> J'ai voulu répondre à une question simple : **la distance routière, ça vaut le coup ?** Parce que c'est cher à calculer.
>
> J'ai fait une ablation propre : **un seul facteur change**, la matrice de distances. Tout le reste est identique. Et toutes les méthodes sont évaluées sur **la même matrice routière**, y compris celles construites en euclidien — sinon l'euclidien gagnerait mécaniquement.
>
> Regardez la première et la deuxième ligne. La distance routière ne fait gagner que **0,002 de silhouette**. C'est invisible. Si je m'étais arrêté là, j'aurais conclu que ça ne sert à rien.
>
> Mais regardez la **dernière colonne** : le temps que chaque enfant passe dans le bus. Là, on passe de 19,2 à 17,5 minutes. **1,7 minute de moins par enfant, tous les jours, soit −9,7 %.**
>
> La leçon : **la silhouette et le temps à bord désignaient deux gagnants différents.** Il fallait décider **avant** ce qu'on optimise. Ce projet optimise le temps passé dans le bus par l'enfant.
>
> Et maintenant, la limite que j'ai trouvée — l'encadré orange. Le nombre de bus était choisi en prenant le **maximum** de silhouette parmi les candidats. Sauf que j'ai mesuré le **bruit** de ces silhouettes : plus ou moins 0,012 à 0,020. Or les écarts entre candidats sont plus petits que ce bruit. **Je faisais donc un maximum sur du bruit.**
>
> Le test qui le prouve : j'ai mélangé l'ordre des mêmes 300 élèves, 30 fois. Résultat : **7 bus dans 57 % des cas, 8 dans 7 %, 9 dans 37 %.** Le nombre de bus ne dépendait pas des données, il dépendait de l'ordre des lignes.
>
> Corrigé : je prends la **médiane** de plusieurs initialisations, et le plus petit nombre de bus qui est à moins d'epsilon du meilleur. Résultat : **7 bus, 30 fois sur 30**. Et les enfants qui changent de bus à chaque inscription passent de 7,2 % à 2,5 %.

Si on te demande : *« Comment avez-vous trouvé ça ? »*

> Pas en relisant le code. En me demandant « qu'est-ce qui se passe si je change juste l'ordre ? ». C'est un test que rien ne m'obligeait à faire.

BLOC E — MS5 : TOURNÉES (1 min)

Slide 14 — Décomposer n'est pas approximer

> **La slide à couper si tu es vraiment en retard.**

À l'écran : le graphique de comparaison à gauche, un tableau à droite, deux encadrés.

Ce que tu dis *(1 minute)*

> Deuxième brique, plus rapidement : l'ordre des arrêts. J'utilise OR-Tools, avec un problème du voyageur de commerce **par bus**, l'école servant de dépôt. Le gain contre un ordre naïf est d'environ **12,6 %**.
>
> Mais je me suis posé une question gênante. Je résous le problème **en deux temps** : d'abord qui monte où, puis dans quel ordre. Or il existe une formulation qui résout les deux **d'un seul bloc** : le CVRP. Et en théorie, décider tout ensemble est toujours meilleur ou égal.
>
> **Alors combien me coûte le fait d'avoir découpé ?**
>
> J'ai fait la comparaison, avec un **budget de calcul égalisé** : mes 7 tournées ont 5 secondes chacune, je donne donc 35 secondes au CVRP pour son calcul unique.
>
> Résultat : **le CVRP ne gagne jamais plus de 0,2 %, et il perd souvent, jusqu'à 2,3 %.**
>
> L'explication est dans l'encadré bleu, et c'est le point à retenir : **mon clustering travaille déjà sur la distance routière.** L'information géographique est donc injectée **avant** le routage. Le CVRP, lui, part de zéro et doit tout redécouvrir dans le même temps.
>
> Donc la décomposition n'est pas une approximation qu'on subit : **c'est une façon d'injecter de la connaissance du domaine dans le solveur.**
>
> Et j'applique ma règle : le perdant est reclassé, pas jeté. Le CVRP a trouvé une solution à **6 bus au lieu de 7**. Il devient donc mon outil de dimensionnement de flotte.

BLOC F — MS12 : PRÉDICTION (3 min)

Slide 15 — Données et variables

À l'écran : six familles de variables à gauche, trois cartes à droite.

Ce que tu dis *(1 minute — tu peux accélérer ici)*

> Troisième brique : prédire l'heure d'arrivée.
>
> J'ai construit **26 variables**, regroupées en six familles. Je commente trois choix.
>
> **L'encodage cyclique de l'heure.** Si je code l'heure de 0 à 23, le modèle croit que 23 heures et minuit sont très éloignés. Je code donc l'heure en **sinus et cosinus**, ce qui les rend voisins.
>
> **Le contexte marocain.** J'ai deux variables qu'on ne trouve dans aucun jeu de données public : le **ramadan** et le **jour de souk**. Le trafic à Meknès n'est pas le même le jour du souk.
>
> **L'historique sans fuite.** J'ai une variable « durée moyenne passée sur ce tronçon ». Le piège classique serait de calculer la moyenne sur toutes les données, y compris le futur. J'utilise donc une moyenne cumulative **décalée d'un pas** : à la date d'aujourd'hui, le modèle ne voit que les jours précédents.
>
> Et un piège que j'ai évité, dans l'encadré bleu à droite : j'ai une colonne « type d'incident » — panne, accident. **Ce n'est pas une variable, c'est une étiquette.** Un incident n'est pas connu à l'heure du départ. Si je l'utilisais comme variable, j'aurais un score magnifique et un modèle inutilisable. Elle me sert uniquement à évaluer, en séparant les trajets normaux des trajets avec incident.

Slide 16 — 11 modèles, un protocole anti-triche

À l'écran : le graphique des MAE à gauche, le tableau à droite.

Ce que tu dis *(1 minute)*

> J'ai comparé **onze modèles**, du plus simple au plus complexe.
>
> Trois précautions de protocole.
>
> **Un — le découpage est temporel, pas aléatoire.** Mon jeu de test, ce sont les **30 derniers jours d'école**. Un découpage aléatoire mettrait des jours futurs dans l'entraînement : le modèle tricherait.
>
> **Deux — une double vue.** Je publie mes métriques sur **tous** les trajets, et séparément sur les trajets **normaux**. Parce qu'un accident qui ajoute 90 minutes n'est pas prédictible : le mesurer, c'est mesurer ma malchance.
>
> **Trois — une baseline métier.** Je ne me compare pas à un modèle bête, je me compare à ce qu'on ferait sans moi : **la durée brute donnée par OSRM**.
>
> Le résultat est en bas du tableau, et il surprend souvent : **c'est ElasticNet qui gagne.** Un modèle linéaire régularisé, devant XGBoost et LightGBM. Erreur moyenne de **1,32 minute**, R^2 de **0,82** sur les trajets normaux, et **52,7 % de mieux** qu'OSRM brut.
>
> Et un point de méthode, dans l'encadré bleu. L'écart avec Ridge est de **2,7 secondes**. J'ai d'abord pensé « c'est du bruit ». J'ai fait un **bootstrap apparié** — c'est-à-dire que je compare les erreurs sur **les mêmes trajets**, pas sur deux échantillons différents. Et là, l'écart est **systématique**, donc significatif. L'incertitude ne va pas toujours dans le sens qu'on croit : il faut la mesurer, pas la supposer.

Slide 17 — Le modèle publié n'était pas celui qui tournait

> **La slide la plus forte du mémoire. Ne la coupe jamais, et ne la survole pas.**

À l'écran : encadré orange (le constat) / encadré bleu (la garde), une phrase-clé au centre, un tableau.

Ce que tu dis *(1 minute — dis-le posément, sans t'excuser)*

> Et maintenant le résultat dont je suis le plus content, même s'il n'est pas flatteur.
>
> Je me suis posé la question de la règle 7 : **quel code s'exécute vraiment quand un parent ouvre l'application ?**
>
> Voilà ce que j'ai trouvé. Mon modèle est entraîné sur des **tronçons de 0,7 à 4 kilomètres** — parce que mon jeu de données découpe les trajets en segments. Mais en service, ce qu'on demande, c'est le **reste de la tournée** : entre 6 et 12 kilomètres.
>
> Autrement dit : **100 % des appels en production sortaient du domaine d'entraînement.** Et mes métriques publiées — le MAE de 1,32 minute — décrivaient un mode qui ne tourne jamais.
>
> Alors j'ai arrêté d'extrapoler. La solution est dans l'encadré bleu : j'interroge le modèle sur un tronçon **de référence**, avec le **contexte réel du jour**. J'en tire un **facteur d'inflation** — entre 1,32 par temps clair et 2,36 sous la pluie avec embouteillage — et je l'applique à la durée qu'OSRM me donne pour le reste de la tournée.
>
> La phrase à retenir est au milieu : **le machine learning dit de combien la circulation se dégrade aujourd'hui, OSRM dit la distance.** Chacun fait ce qu'il sait faire.
>
> Et j'ai mesuré cette garde sur des trajets entiers. Erreur de **2,17 minutes** avec la garde, contre 3,45 en extrapolant — donc **37 % d'erreur en moins** — et 9,33 pour OSRM seul, donc **77 % en moins**.
>
> Dernier point, l'encadré du bas. J'ai fait une ablation famille par famille, et deux résultats m'ont surpris. La variable « historique » est ma **7e importance publiée** — mais la retirer coûte **0,003 minute**, c'est-à-dire rien. Et retirer les variables de temps d'arrêt **améliore** le modèle. Conclusion : **l'importance dit ce que le modèle utilise, seule l'ablation dit ce qu'il perdrait.** Ce n'est pas la même chose.

Si on te demande : *« Pourquoi ne pas réentraîner sur des trajets longs ? »*

> C'est la bonne solution, et elle est dans mes perspectives. Il faudrait régénérer le jeu de données au niveau du trajet complet. J'ai choisi de ne pas toucher à l'artefact gelé pendant la phase de mesure, pour que les chiffres publiés restent comparables.

BLOC G — MS11 : ANOMALIES (3 min)

Slide 18 — Deux horizons de surveillance

À l'écran : deux colonnes, A et B, et un encadré bleu en bas.

Ce que tu dis *(50 secondes)*

> Quatrième brique : détecter ce qui est anormal. Et j'ai **deux dispositifs**, pas un — c'est le point que beaucoup de projets confondent.
>
> **À gauche, les règles instantanées.** Elles s'appliquent à **chaque point GPS**, toutes les cinq secondes : vitesse au-dessus de 90, arrêt de plus de 5 minutes, et écart de plus de 250 mètres par rapport à la tournée prévue.
>
> **À droite, l'Isolation Forest.** C'est un modèle de machine learning **non supervisé** : il n'apprend jamais les étiquettes. Il travaille sur des **trajets complets**, après coup.
>
> Pourquoi les deux ? Parce qu'**un modèle qui note un trajet complet ne peut pas juger un point GPS isolé**. Les deux répondent à deux questions différentes.
>
> Et en bas, un choix important : j'ai construit **trois natures d'anomalie** volontairement différentes. Le **franc** — une panne, avec un rapport énorme. Le **contextuel** — un retard modéré, mais par beau temps et trafic fluide, donc anormal. Et la **surcharge** — un temps normal, mais un nombre de montées aberrant. Aucun seuil simple ne peut capter les trois. C'est ça qui rend le test honnête.

Slide 19 — Le feature engineering prime sur l'algorithme

À l'écran : le graphique PR-AUC à gauche, le tableau à droite.

Ce que tu dis *(1 min 05)*

> J'ai comparé quatre détecteurs, avec un **budget d'alerte égalisé** : chacun a le droit d'émettre exactement le même nombre d'alertes. Sinon celui qui alerte tout le temps aurait un meilleur rappel, mécaniquement.
>
> Je mesure en **PR-AUC**, pas en accuracy. Avec 1,25 % d'anomalies, un modèle qui dit toujours « normal » a 98,75 % d'accuracy. L'accuracy n'a aucun sens ici.
>
> Regardez la première et la dernière ligne. **0,9127 contre 0,4818.** C'est le **même algorithme**, les mêmes hyperparamètres, la même graine aléatoire. La seule différence : **6 variables choisies au lieu de 25**.
>
> Autrement dit, **la curation des variables vaut plus que le choix de l'algorithme.** Les 19 variables en trop diluent le signal.
>
> Et la variable qui fait la différence est en bas à droite. Je l'appelle `surprise`. Ce n'est pas le retard brut, c'est **le retard divisé par le retard qu'on attendait vu le contexte**. Un retard de 40 % sous la pluie aux heures de pointe, c'est banal. Le même retard par temps clair et trafic fluide, c'est anormal. **C'est cette variable-là qui attrape les anomalies contextuelles**, celles qu'aucun seuil brut ne voit.

Slide 20 — Deux mesures qui contredisent mon propre dossier

> **Slide à ne jamais couper.**

À l'écran : deux encadrés orange, chacun avec un petit tableau.

Ce que tu dis *(1 min 05)*

> Et maintenant, deux mesures qui contredisent ce que j'avais moi-même publié.
>
> **La première, à gauche.** Mon verdict gelé disait : l'Isolation Forest gagne, PR-AUC 0,91 contre 0,80 pour le seuil simple. Sauf que ce verdict repose sur **8 anomalies de test**. Huit.
>
> J'ai fait un bootstrap : les intervalles de confiance **se recouvrent largement**. Puis j'ai fait une **validation glissante** sur quatre blocs, ce qui porte l'effectif à **38 anomalies**. Et là, **le classement s'inverse** : le seuil univarié passe devant, 0,9262 contre 0,8465.
>
> Donc ma thèse « il faut du multivarié » est **fausse** telle que je l'avais écrite. Le seuil attrape mieux les anomalies contextuelles ; l'Isolation Forest, lui, est le seul à attraper les surcharges. Ils sont complémentaires, pas hiérarchisés.
>
> **La deuxième, à droite, est plus embêtante.** J'ai rejoué mes règles temps réel sur une **vraie tournée de 753 points**. Et j'ai découvert que ma règle de déviation est largement aveugle.
>
> La raison est géométrique : la règle mesure la distance au point le plus proche de **toute** la polyligne. Or une tournée de 50 arrêts **repasse dans le même quartier**. Donc un bus qui a fait un détour de 250 mètres reste souvent proche… d'une autre portion de sa propre tournée.
>
> Le chiffre : un détour de 250 mètres n'est détecté que sur **9,6 % du parcours**.
>
> Et j'ai la solution mesurée : en abaissant le seuil à 120 mètres, **la couverture double, sans aucune fausse alarme**, même avec 30 mètres de bruit GPS.
>
> Ces deux défauts sont **rapportés et chiffrés, pas corrigés**. Corriger un seuil en production est une décision d'exploitation, pas une décision de mémoire.

BLOC H — MS9 : ASSISTANT (3 min)

Slide 21 — Un agent qui n'a le droit de voir que vos enfants

> **La slide à couper en priorité dans ce bloc** (garde 22 et 23).

À l'écran : quatre étapes en haut, la liste des outils à gauche, un encadré bleu à droite.

Ce que tu dis *(1 minute)*

> Cinquième brique : un assistant conversationnel pour les parents.
>
> Le principe, c'est le **function calling** : le parent pose une question, le modèle de langage choisit lui-même quel outil appeler, l'outil interroge un microservice, et le modèle rédige la réponse avec les faits obtenus.
>
> J'ai écrit **cette boucle à la main**, sans LangChain. Ce n'était pas de l'orgueil : je voulais que **le contrôle de sécurité soit dans mon code**, et que je puisse le montrer.
>
> J'ai cinq outils : les enfants du parent, la position du bus, l'heure d'arrivée prévue, le trajet en cours, et les incidents. Le premier passe même par un géocodage inverse, donc l'assistant peut répondre « votre bus est rue Zitoune » au lieu de donner des coordonnées.
>
> Et le point critique est dans l'encadré bleu. La liste des bus autorisés est **dérivée de l'identifiant du parent contenu dans le jeton** — **jamais** des identifiants proposés par le modèle. Parce qu'un modèle de langage à qui on dit « et le bus numéro 3 ? » propose très volontiers 3. L'outil vérifie que ce bus appartient à celui qui demande, **avant tout appel réseau**.
>
> C'est la règle : **on ne fait jamais confiance à ce que le modèle propose comme identifiant.**

Slide 22 — NL2SQL : trois couches

À l'écran : trois cartes en haut, un tableau à gauche, un encadré orange à droite.

Ce que tu dis *(1 minute)*

> J'ai ensuite fait une deuxième phase, pour l'administrateur : **poser une question en français, et obtenir la réponse depuis la base de données**. Le modèle génère du SQL, on l'exécute, et il rédige la synthèse.
>
> Faire écrire du SQL par un modèle de langage et l'exécuter sur une base de production, c'est évidemment dangereux. J'ai donc **trois couches**.
>
> **Première couche** : un filtre par expressions régulières, qui refuse les mots dangereux. Il bloque 14 attaques sur 14, sans casser aucune des 8 requêtes légitimes.
>
> **Deuxième couche** : la connexion utilise un **rôle PostgreSQL dédié, en lecture seule**. Donc même si un `DROP TABLE` passait le filtre, **le serveur le refuserait**.
>
> **Troisième couche** : la requête tourne dans une transaction en lecture seule, avec un délai maximal de 5 secondes et un plafond de résultats.
>
> Les mesures : **83 % d'exactitude stricte, 94 % en tolérant les colonnes supplémentaires**, et **zéro fuite sur six attaques**.
>
> Et je veux commenter l'encadré orange, parce que c'est mon résultat le plus inconfortable et le plus utile. **Aucune des six attaques n'a atteint mon filtre ni PostgreSQL.** Le modèle les a refusées lui-même, avant. Donc le « 100 % sans fuite », **il est obtenu par le modèle, pas par mon architecture**.
>
> Et c'est exactement pourquoi les couches profondes doivent exister. Un modèle plus faible, ou une injection mieux construite, franchirait la première couche. **On ne peut pas fonder une sécurité sur la docilité d'un modèle de langage.**

Slide 23 — Évaluer un agent : 36 questions, 3 langues, 5 modèles

À l'écran : un tableau de 5 modèles, et deux encadrés en bas.

Ce que tu dis *(1 minute)*

> Comment évalue-t-on un agent conversationnel ? J'ai construit un **jeu de test versionné de 36 questions**, en six catégories : questions opérationnelles, questions sur l'école, hors-sujet, **tentatives d'injection**, questions sur l'enfant d'une autre famille, et demandes de modification.
>
> Et en **trois langues : français, arabe, et darija marocaine**. La darija n'est pas un ornement : c'est la langue dans laquelle les familles de Meknès poseront réellement leurs questions.
>
> Précaution de protocole : j'ai **neutralisé la cascade** pendant la mesure. Sinon, un modèle saturé serait silencieusement remplacé par le suivant, et je mesurerais un mélange.
>
> Les résultats. Mon modèle principal fait **100 % sur les quatre critères**. Le modèle de repli fait 88 %.
>
> Mais l'encadré bleu contient le résultat le plus utile : **le modèle de repli se dégrade spécifiquement en darija**. Il redemande de quel enfant il s'agit au lieu de répondre. Donc le premier étage de ma cascade compte surtout **pour les familles qui n'écrivent pas en français** — c'est-à-dire le cœur de ma cible.
>
> Et l'encadré orange : **ce benchmark a trouvé une panne de production.** Regardez la dernière ligne. Le troisième et dernier étage de mon repli, `gemini-2.0-flash`, répondait **404 sur les 36 questions**. Le modèle n'existe plus, il a été retiré par le fournisseur. Ma clé était valide. Depuis une date inconnue, mon assistant **n'avait plus aucun filet de sécurité**.
>
> Ce défaut était invisible tant qu'un des deux étages principaux répondait. Il se serait manifesté le jour d'une saturation — statistiquement, le jour de la démonstration.
>
> La leçon est architecturale : **dans une cascade de repli, viser un alias, jamais une version figée.** Un alias survit aux renouvellements de génération.

BLOC I — BILAN (1 min 50)

Slide 24 — Ce qui est vérifié, et ce qui ne l'est pas

À l'écran : deux colonnes, verte à gauche, orange à droite, et trois perspectives en bas.

Ce que tu dis *(1 min 30)*

> Pour finir, je veux être précis sur ce que j'ai vérifié et ce que je n'ai pas vérifié.
>
> **À gauche, ce qui est mesuré et rejouable.** J'ai écrit une recette de bout en bout, en huit phases, qui tourne contre le système en fonctionnement. 88 tests d'API, 67 vérifications de contrats et de rôles, 39 scénarios entre acteurs, 35 assertions sur la justesse des briques IA. Et un balayage automatisé des 27 écrans de la console dans un vrai navigateur Chrome, pour les trois rôles.
>
> Tout est rejouable par une commande. Un chiffre que personne ne peut rejouer cesse d'être une garantie.
>
> **À droite, ce que je n'ai pas vérifié, et que j'assume.**
>
> Je n'ai **aucune donnée terrain** : deux de mes briques reposent sur des générateurs, avec une seule graine aléatoire. Je travaille sur **une seule instance** : une école, 300 élèves.
>
> Deux de mes services Python **n'ont aucun test unitaire**. Et côté Java, `mvn test` n'exécute que **18 tests** au lieu des 22 que j'annonçais, parce que cinq modules ont les tests désactivés. Je l'ai découvert pendant la recette — un test désactivé est pire qu'un test absent, parce qu'il rend le rapport vert.
>
> Enfin, mon WebSocket GPS n'est pas authentifié. C'est une dette identifiée, que je n'ai pas traitée : la corriger touche les trois frontends d'un coup.
>
> Mes trois perspectives sont en bas. La première est la plus importante : **réentraîner le modèle de prédiction sur les durées réelles** que le système collecte maintenant lui-même. C'est ce qui lèverait la circularité que je vous signalais au début.

Slide 25 — Merci

À l'écran : « Merci de votre attention », la carte des 7 groupes de production.

Ce que tu dis *(20 secondes)*

> Voilà. 300 élèves, 7 bus, et cinq briques de science des données qui tournent réellement en production — mesurées, et rejouables.
>
> Je vous remercie de votre attention, et je suis à votre disposition pour vos questions.

ANNEXES (slides 26 à 32)

> **Ne les présente pas.** Elles servent uniquement à répondre. Quand une question arrive, dis « j'ai le détail en annexe » et saute directement à la bonne slide.

Slide	Contenu	Question à laquelle elle répond
27	Les 11 modèles de prédiction, métriques complètes	« Pourquoi ElasticNet et pas XGBoost ? »
28	Les 4 détecteurs d'anomalies + intervalles de confiance	« Votre PR-AUC de 0,91, elle est fiable ? »
29	L'arène complète du clustering, 6 méthodes × 8 métriques	« Comment savez-vous que votre méthode est la meilleure ? »
30	Le détail des variables construites	« Quelles variables exactement ? »
31	La topologie événementielle complète	« Comment les services communiquent-ils ? »
32	Le corpus de sécurité SQL, 29 cas	« Comment savez-vous que votre filtre SQL tient ? »

Les questions les plus probables, et quoi répondre

« Vos données sont simulées, donc vos résultats ne valent rien. »

> Une partie des données est réelle : les 300 élèves, leurs coordonnées, et tout le réseau routier via OSRM. C'est sur ces données réelles que reposent le clustering et les tournées.
>
> Pour la prédiction et les anomalies, oui, c'est simulé — et je l'ai dit dès le début. Mais ce qui se transporte à des données réelles, ce n'est pas le chiffre, c'est **le protocole** : le découpage temporel, le bootstrap apparié, l'ablation, les baselines métier. Ces mesures seraient identiques demain sur des données terrain. C'est d'ailleurs ma première perspective.

« Pourquoi 13 microservices pour 300 élèves ? C'est disproportionné. »

> Vous avez raison, et je l'assume. Pour ce volume, une application unique aurait suffi. Deux raisons expliquent le choix : j'ai besoin de deux écosystèmes, Java pour les transactions et Python pour scikit-learn et OR-Tools ; et je voulais que le clustering, qui prend plusieurs secondes, ne bloque jamais le reste. Le vrai bénéfice a été ailleurs : chaque brique IA peut être benchmarkée et redéployée seule.

« Votre silhouette est à 0,43, c'est faible. »

> C'est un arbitrage, et il est chiffré. Sans contrainte de capacité, j'atteins 0,4636 — mais avec un groupe de 66 élèves, donc infaisable. Le réalisme me coûte 3,4 %. Et j'ajoute que sur ma métrique métier, le temps passé dans le bus par enfant, ma solution est meilleure que celle qui a la meilleure silhouette.

« Quel est votre apport personnel, par rapport à un assemblage de bibliothèques ? »

> Trois choses. **Un** : les décisions de préparation — la projection UTM et la matrice routière — qui ne sont dans aucune bibliothèque et qui changent le résultat de 9,7 % sur ma métrique métier. **Deux** : la garde anti-extrapolation, qui vient du constat que mon modèle publié n'était pas celui qui tournait. **Trois**, et c'est le principal : **le protocole de benchmark en sept règles**, qui a contredit une affirmation de mon propre dossier dans les cinq briques sur cinq. Assembler des bibliothèques, tout le monde sait le faire. Trouver que ce qu'on a publié est faux, c'est plus rare.

« Que feriez-vous avec six mois de plus ? »

> Trois choses, dans cet ordre. Réentraîner la prédiction sur les durées réelles maintenant collectées. Corriger la règle de déviation en utilisant l'abscisse curviligne plutôt que la distance au point le plus proche — j'ai déjà mesuré que ça marcherait. Et livrer la réaffectation dynamique, la seule exigence du cahier des charges restée ouverte.

« Pourquoi avoir passé du temps à casser votre propre travail ? »

> Parce qu'un chiffre flatteur sans protocole ne prouve rien, alors qu'un chiffre décevant obtenu proprement est une contribution. Et concrètement, ça a payé : cette campagne a trouvé une panne de production réelle — mon repli d'assistant était mort — et un défaut de stabilité qui aurait fait changer de bus une trentaine d'enfants à chaque inscription.

Trois conseils pour le jour J

1. **Ne lis pas tes slides.** Le public sait lire. Ta valeur ajoutée, c'est ce qui n'est pas écrit : pourquoi tu as choisi ça, et ce que tu as appris en te trompant.
2. **Assume les chiffres moyens.** La silhouette à 0,43, les 9,6 % de couverture de la déviation, le 404 de ton modèle de repli : ce sont tes **meilleurs** moments, pas tes pires. Un étudiant qui trouve les faiblesses de son propre travail avant le jury a déjà gagné la discussion.
3. **Garde le mot « mesuré » dans la bouche.** Chaque fois que tu affirmes quelque chose, dis comment tu l'as vérifié. « J'ai mesuré », « j'ai rejoué 30 fois », « le bootstrap dit que ». C'est ce vocabulaire-là qui distingue un master en science des données d'un projet de développement.